Project: Genetic Algorithm

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: GPU Computing

June 15, 2026

Contents

1. Intro

This report presents “Project: Genetic Algorithm” assignment for the GPU Computing course. It begins by establishing the context of applying a Genetic Algorithm to the specific problem space in Section 2. Section 3 then outlines the data preparation pipeline and provides further details on the problem itself.

Following, the updated sequential implementation and initial naive speedup methods are detailed in Section 4. Finally, the report focuses on the parallel implementation discussed in Section 5 — before concluding with a comparative performance analysis of both versions in Section 6.

FIXME: Make sure this is present in the final version

1.1. Declaration of AI Usage

For starting the project and the start up phases AI was used. The repository for preprocessing the DEM data can be found here [1]. As a starting point, the following repository was used for both the sequential & parallel version of the algorithm: [2]. The initial repository was updated to reflect the modern features, such as the deprecated `rand()` function and more. After the code was updated, some AI help was used to think through the genetic algorithm structures.

2. Context

The problem addressed by this genetic algorithm is a well-known challenge in wireless networking: optimizing service tower placement to maximize coverage, while accounting for terrain variations [3]. Specifically, wireless internet providers must strategically position their infrastructure to ensure optimal signal distribution across complex geographic landscapes.

An identical problem arises within the military domain, where a limited set of sensors must be strategically deployed to maximize total coverage or detection probability [4].

For these very reasons, there have been historically been several successful attempts in applying Genetic Algorithms to solve this particular problem [5].

A notable variation of this problem involves the deployment of two distinct sensor types, Forward-Looking Infrared (FLIR) and seismic, across hilly terrain to detect approaching military vehicles [6]. In this scenario, the algorithm is utilized to optimize the placement strategy for both sensor types.

Consequently, this paper implements a scoped-down version of the genetic algorithm presented in [6]. The implementation focuses exclusively on a single sensor type: a Forward-Looking Infrared (FLIR) sensor positioned 1.8 m above the ground—as illustrated in Figure Figure 5.

3. Pipeline & Genetic Algorithm

This section will discuss the pipeline of preparing and retrieving required DEM (Digital Elevation Model) data, and the outputs the python preprocessing scripts generated for use in the genetic algorithm.

3.1. Overview

The complete pipeline can be viewed in Figure 1.

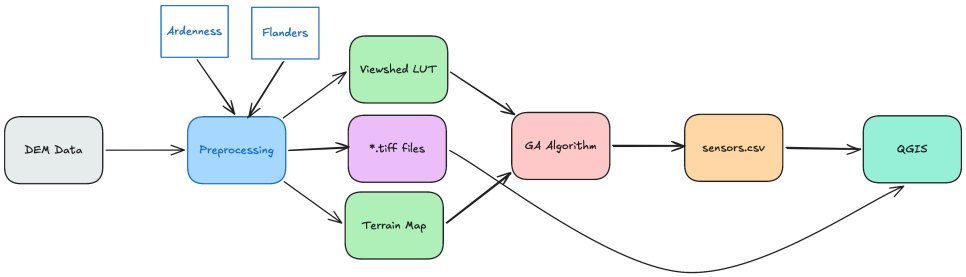

Figure 1: Pipeline Overview

The process starts with downloading the DEM data from a public source. Once the data is downloaded, the preprocessing step, based on the specified range, generates a Viewshed LUT (Look-Up-Table), Terrain Map and other `*.tiff` files. The viewshed LUT, is a 4D array: `viewshed[sx][sy][tx][ty]` , consisting of ones and zeros, based on if source & target can see each other. The terrain map, is a 2D array which includes the elevation of the particular cell in question.

After the viewshed & terrain files are generated, the genetic algorithm is executed with those files as input. Based on the provided parameters, the GA computes an optimal solution and generates a `sensor.csv` file. The file, containing sensor position based on grid coordinates.

The `*.tiff` files are created for visualization purposes and can be imported into QGIS[1]. The generated `sensors.csv` file can subsequently be converted for further analysis into GEO spatial coordinates file called: `geo_sensors.csv` using `convert_sensors.py` , which can be imported into QGIS.

3.2. DEM

The genetic algorithm itself requires appropriate terrain data for a more realistic scenario. This type of data can be easily downloaded from public sources. Therefore, the data for the country of Belgium was used. In this case, the source of the data were obtained from OpenTopography.org[2] and was downloaded on 8 April 2026.

The two areas selected as a comparison of the GA between flat terrain and a bit more hilly terrain can be seen in Figure 2 & Figure 3.

[1] https://qgis.org

[2] https://portal.opentopography.org/raster?opentopoID=OTSDEM.032021.4326.2

Figure 2: Ardennes Geolocation Area

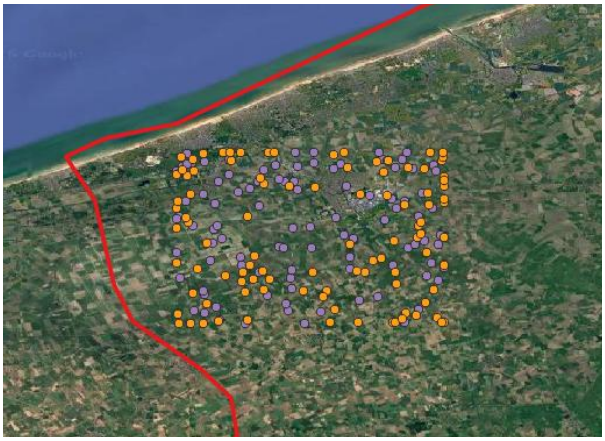

Figure 3: Flanders Geolocation Area

The coordinates of both areas are the following: (Gdal format):

- Ardennes: 5.4, 49.8, 5.540, 49.890
- Flanders: 2.58, 51.00, 2.72, 51.09

3.3. Preprocessing

Once the DEM data is available, the preprocessing step using `python` may begin. For this type of data & problem, there is a library available that handles the heavy preprocessing lifting that is required, called: `gdal` [7].

The repository, which includes the preprocessing files, can be found at [1]. A short description and workings of the preprocessing will be made here. The repository consists of the following files: `preprocess_dem.py` , `preprocess_viewshed.py` , `convert_sensors.py` , `check_sizes.py` , `check_visbility.py` .

The `preprocess_dem.py` is responsible for converting a given square area based on ROI (Gdal coordinates) into a binary file containing area elevation data. The script creates several output files, the 2 most important files for the GA are: `elevation.bin` and `elevation_meta.txt` . The remaining output files were used to verify the GA algorithm in QGIS, as shown in Figure 2, Figure 3.

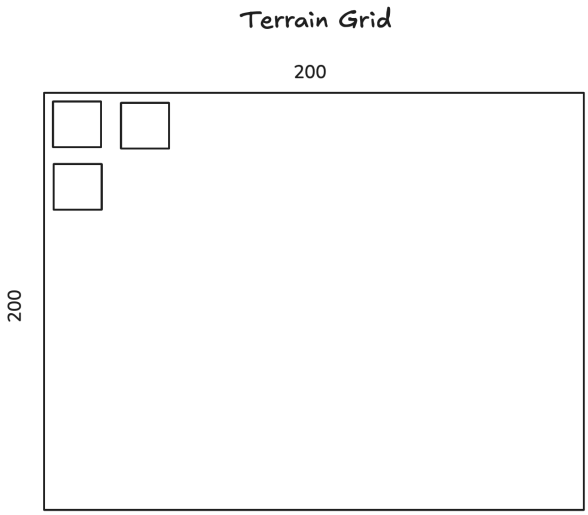

Figure 4: Terrain Elevation Grid

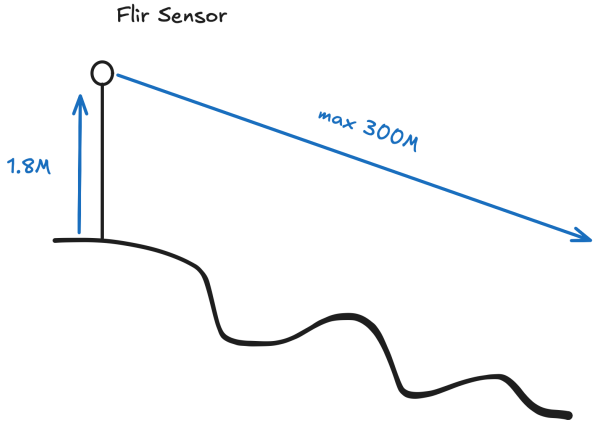

Figure 5: Terrain Sensor

The viewshed is generated by the `preprocess_viewshed.py` file. It uses the `gdal.ViewshedGenerate` function to calculate the visibility of a source sensor 1.8m high and target 0m high (on the ground). Depending on the problem, these parameters can be tuned. This script creates several files, two files are used as input for the GA: `viewshed_lut.bin` and `viewshed_meta.txt` .

There are two verification files included: the first one is `check_sizes.py` : which checks the size of each cell depending on the size of the area and the number of cells of the grid. The second one: `check_visibility.py` uses Gdal functions to check the general visibility of the area by reading in the `viewshed_lut.bin` file.

To complete the list of scripts: `convert_sensors.py` is responsible for converting the `*-sensors.csv` file generated by the GA algorithm from grid based coordinates to GEO based coordinates for import and visualization in QGIS.

4. Sequential

This section will discuss the updates made to the sequential implementation of a genetic algorithm and a naive attempt at parallelizing the implementation by using OpenMP[3].

4.1. Implementation

The algorithm's generation loop begins by calling the `generateOffspring` function, which scales up the fitness value of all chromosomes by a large multiplier (1000000). The chromosome list is then iterated over to select pairs of parents. For each selection, a roulette wheel mechanism is used. If the roulette value is lower than a randomly generated threshold, a parent is chosen; otherwise, a random index is returned.

Both parents are passed to the `crossover` function, for which both parents in the `population` are retrieved and both child values from the original chromosome in the `buffer` population. A random cross over point is selected using `rng`. The list of genes (sensors) is iterated. Once the cross over threshold is reached, the sensor position values are swapped, before the threshold are copied.

The next step in the flow is mutating of 2 consecutive chromosomes by two `mutate` calls. They respectively iterate the list of genes and mutate genes values (x,y) coordinates by a random delta. Mutation probability is decided by a RNG based `prob_dist` and `delta_dist`.

The mutated population is evaluated by the `evaluate` function. All genes of each chromosome are evaluated by the `computeChromosomeFitness` function. First, shared sensor values are calculated. Proceeding, `GRID_SIZE` x `GRID_SIZE` cells are iterated. For each cell, the visibility factor is checked using `getVisibilityFactor`, the POD table is used to look up POD using `lookupPOD`.

If the population has not reached the indicated maximum convergence value, the generation loop will continue.

4.2. OpenMP

Several `for` loop inside of the code have received a `#pragma` for enabling acceleration with earlier named OpenMP library. Due note that this is a naive application of the use of OpenMP pragma's, and is an indication of how a naive application compares to a more tailored made parallel genetic algorithm using Cuda.

[3] https://www.openmp.org/

4.3. Results

This subsection will showcase the results of applying OpenMP pragma's on the genetic algorithm. The result can be seen in Figure 6.

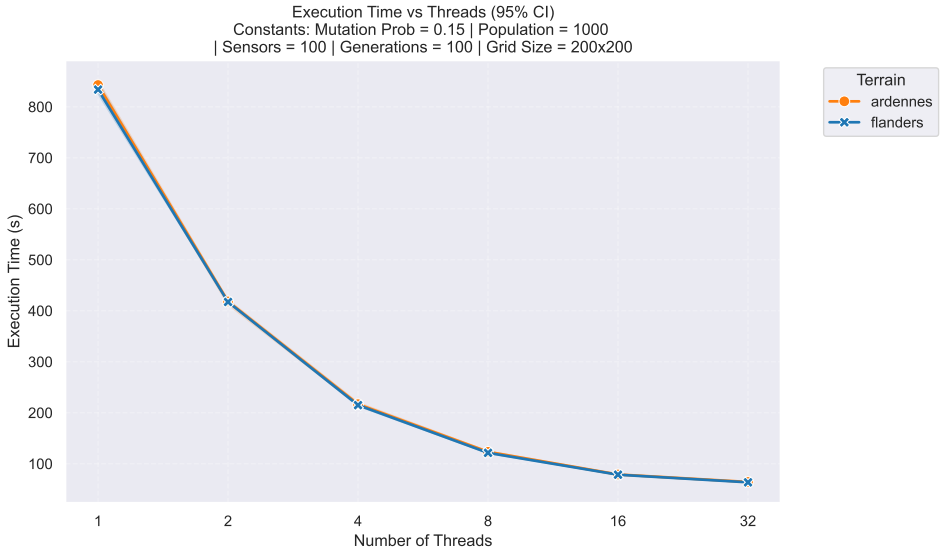

Figure 6: Sequential GA - OpenMP

As can be seen in the image, the addition of even naive application of OpenMP pragmas results in a **8x** reduction in execution_time between the single thread variant in the *32* thread variant.

Expanding the analysis to the fitness value and speedup, both are shown in Figure 7.

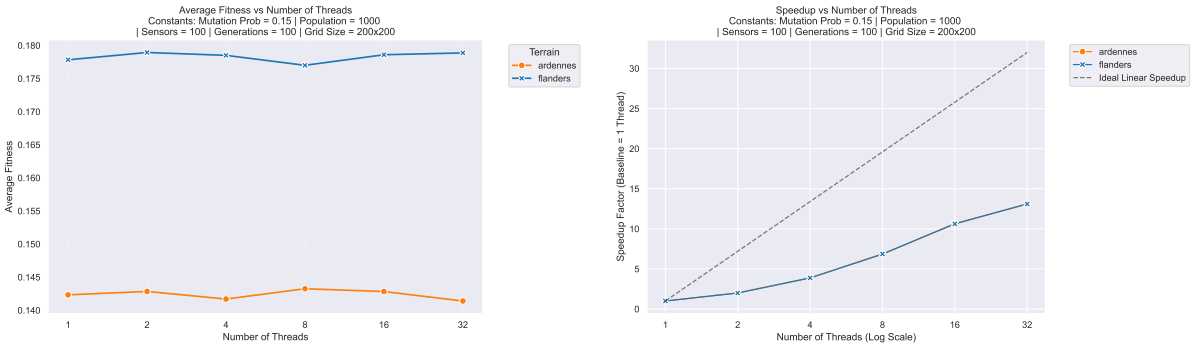

Figure 7: Fitness (right) and Speedup (left)

The left image shows that each terrain has a region of fitness and that increasing the number of threads allows the fitness values to fluctuate. However, no significant increase or decrease is noticed.

The right chart, showcasing a speedup based on the 1 thread execution time, indicates the successful application of multiple threads to decrease execution time, while not having to fundamentally change the algorithm.

5. Parallel

This section will discuss the creation & implementation of the Genetic algorithm for computing using Cuda. First, the genetic structure of the algorithm will be discussed in Section 5.1, a list of kernels & helper functions in Section 5.2. In Section 5.3, a flowchart illustrating the algorithm execution. To complete, a detailed analysis of each kernel in Section 5.4.

5.1. Genetic Structure

The genes of the genetic algorithm are structured as `double` value, as shown in Listing 1.

```
1 typedef struct
2 {
3     double val;
4 } gene_t;
```

Listing 1: Gene Structure

Each gene will contain one of the axis of a sensors coordinates: (x, y). The gene's (sensor) location are stored in one continues list, as shown in illustration Figure 8.

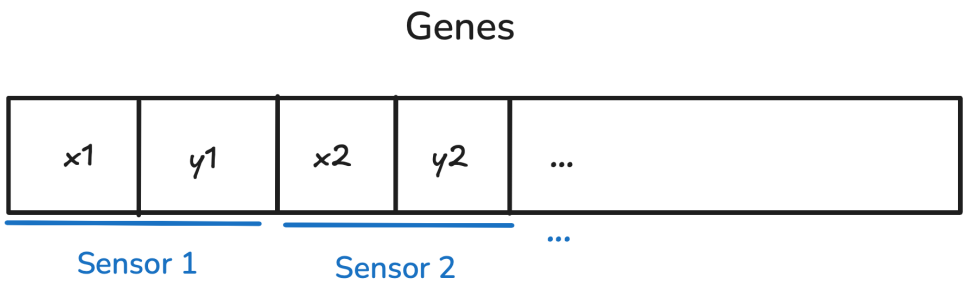

Figure 8: Genes Structure

Continuing on from the genes, the chromosomes of the algorithm have the following structure, as shown in Listing 2.

```
1 typedef struct
2 {
3     double fitness;
4     int geneIdx;
5 } chromosome_t;
```

Listing 2: Chromosome Structure

Each individual chromosome has a `double` fitness value associated with itself and `geneIdx`, which is the starting index of where the genes for that chromosome begin in the list. Combining both genes & chromosome structure, result in the following organization as illustrated in Figure 9.

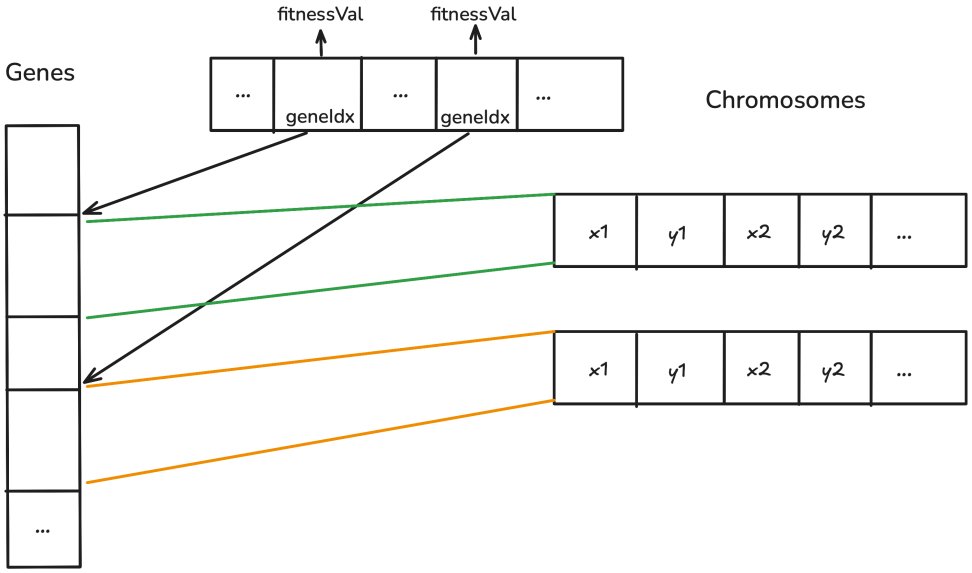

Figure 9: Genes & Chromosome Structure

The `geneIdx` value is an offset within the list of overall genes to which the chromosome stores its responsible genes. Based on this offset, the sensor location can be accessed. Each chromosome maintains a `fitnessValue` indicating the fitness value for his list of genes.

Walking up the ladder of structures of the algorithm, the over arching one is the `population_t` structure, as illustrated in Listing 3.

```
typedef struct
{
    // FLIR range parameters
    (...)

    // Terrain parameters
    (...)

    // GA parameters
    (...)

    // Chromosomes & Genes
    chromosome_t *chromosomes;
    gene_t *genes;

    // GPU-side data tables
    double *pod_table_sq;
    float *terrain_elevation;
    uint32_t *packed_terrain_viewshed;
} population_t;
```

Listing 3: Population Structure

The structure stores the FLIR parameters, terrain parameters, GA parameters (mutation, etc), the chromosomes & genes list, the pod table, terrain elevation data and viewshed LUT.

5.2. Kernels Overview

This subsection will briefly discuss the list of implemented kernels and their purposes before diving deeper into the following section.

The following is the list of specific kernels:

- `initBufferKernel` : initializes chromosome values with default values for a population
- `setupCurand` : seeds a list randomness values in a shared state object
- `initIslandBufferKernel` : initializes per island chromosome values with defaults for a population
- `buildRouletteKernel` : initializes roulette values for each island
- `gaIslandKernel` : per island GA: `roulette` , `crossover` , `mutate`
- `migrateRingKernel` : migrates population between islands, shared ring buffer
- `evaluateIsland` : evaluates each island's list of chromosomes

The next is a list of helper kernels which are executed by the preceding list of kernels:

- `cudaInitPopulation` : initializes each island population
- `rouletteSelect` : performs selection of chromosome
- `crossover` : performs genes crossover between current & new population
- `mutate` : mutates the genes for selection chromosome
- `getVisibilityFactor` : retrieves the visibility factor based on source & target coordinates
- `lookupPODSq` : POD (Probability of Detection) lookup
- `calculateOverlapPenalty` : calculates penalty if sensor overlap
- `calculateCellPOD` : calculate sensor POD based on sensor coordinates

5.3. Flow

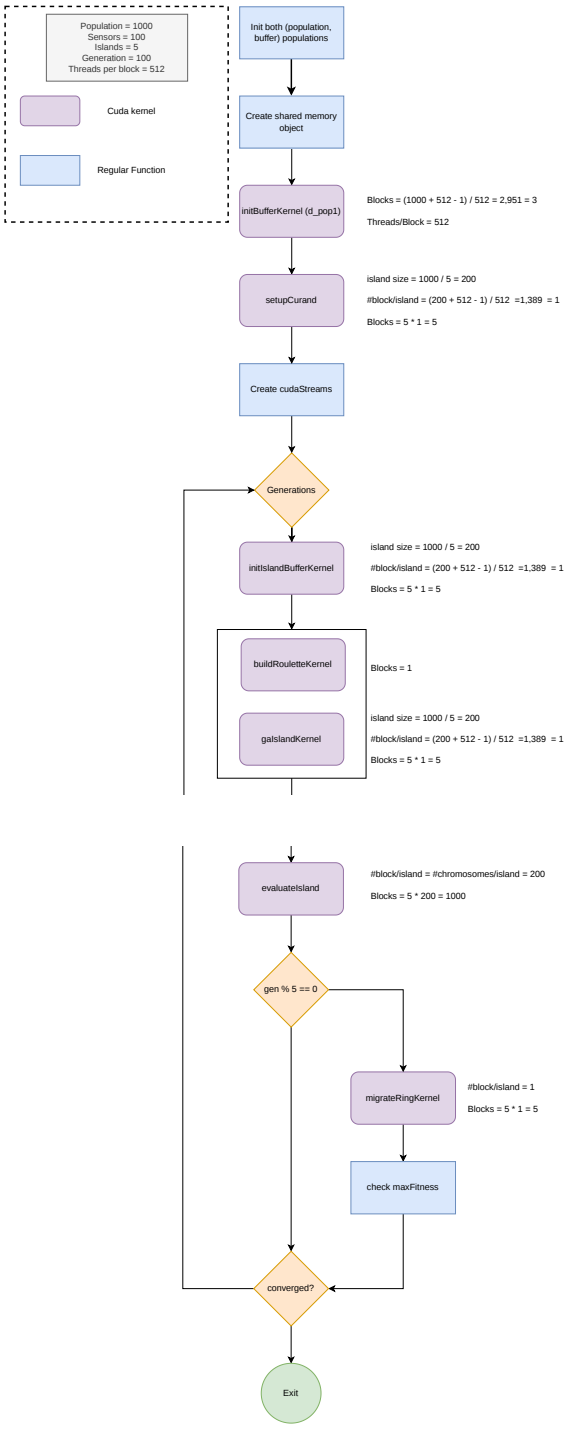

Figure 10: Genetic Algorithm Flow

The explanation of the algorithm flow will use the following parameters:

- population size: 1000
- sensors: 100
- islands: 5
- generation: 100
- #threads/block: 512

The genetic algorithm starts of with initializing two populations, a `buffer` and main `population`. The `cudaInitPopulation` is the function where the CPU memory is copied into the GPU's memory.

Once the buffers have been initialized, the size of the `sharedMemSize` object is calculated for later use in the `evaluateIsland` kernel. The `d_pop1` is set to `cudaPopulation` and `d_pop2` is set to `cudaBuffer`.

The `d_pop1` kernel is initialized using `initBufferKernel`. For the example parameters, this will result in 3 blocks being launched.

Each island requires a `curand` state. This initialization is handled by the `setupCurand` kernel, for the example, 5 blocks will be launched.

In an attempt at speeding up the execution of the evaluation of the chromosomes, `cudaStream`'s[4] were used. The number of streams equals the number of islands.

After the setup, the generation loops initiates. The first step is the `initIslandBufferKernel`, it will initialize each kernel separately in the `d_pop2` population variable. Each separate island may now execute the GA algorithm independently after initialization.

Before the GA can be executed, the roulette selection must be setup by `buildRouletteKernel`. This is a single block & single thread execution.

Each `galsland` kernel will launch 1 block, and in total 5 blocks will be executed. Each thread will perform some iterations to select some chromosomes for selection.

[4] https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html

Selection is performed by `rouletteSelect`. For each pair of selected parents, `crossOver` is applied to generate offspring. The resulting offspring then undergoes mutation of its (x,y) coordinates through `mutate`. The mutated individuals are written to the `buffer` population. Finally, the `threadState` is stored in the global states.

Once the population of each kernel is 'updated', each island can be evaluated for its fitness. Before doing so, a pointer swap is performed. The previous `buffer` population (`d_pop2`) is swapped with `d_pop1`, the latter value is evaluated.

Each island and population is evaluated separately `evaluateIsland`, for each island, 200 blocks are spawned, matching the number of chromosomes.

Global migration between the island only occurs every **5** generations, as set by the `migration_interval`. This allows each island to independently grow, without incurring migration cost every generation. At the same time, every **5** generations, the chromosomes are checked to see if any have reached convergence.

If the `maxFitness` value has not reached the convergence value, the generation execution continues.

5.4. Kernels

This subsection will discuss some of the kernels in more depth.

5.4.1. `gaIslandKernel`

Each island is responsible for executing the genetic algorithm in isolation. The genetic algorithm includes the same steps as with the sequential version.

Since each island is constrained to an island, it has to work of the main array of chromosome. Each island has a size based on the number of island and population size (#chromosomes).

For each island, each thread is responsible for generating several pairs `(x,y)` of offspring, the offspring is selected by calling `rouletteSelect`. The global parent coordinates are found by adding the island offset. Than, the `crossOver` step is executed using parent indices, followed by two `mutate` calls on the `buffer` population.

5.4.2. `evaluateIsland`

The next step after the GA algorithm has been applied on each respective island in order to check the fitness of each island's chromosomes. For this particular kernel, the number of blocks launched, matches the island size (#chromosomes).

The first step in the evaluation process, is to populate the shared data object. The object `extern __shared__ char rawSharedData[]` is initialized by a single thread. Once the shared data object has been populated, the grid cell loop can start.

For each cell in the grid ($200 * 200$) `calculateCellPOD` function is called. The function iterates the list of sensors – calculates the visibility factor between sensor & target coordinates – applies overlap penalty as needed and returns the combined POD value.

The localPOD is added to a `blockSum` variable indexed by `threadIdx.x`. The array of local POD values is than summed by using parallel reduction on the `blockSum` array.

The thread with id 0, will divide the POD by the number of grid cells. The resulting fitness value is written to the `fitnessOut` value, an array of all chromosome fitness values.

5.4.3. `migrateRingKernel`

The configuration of the migration of chromosomes between the islands can be seen in Figure 11.

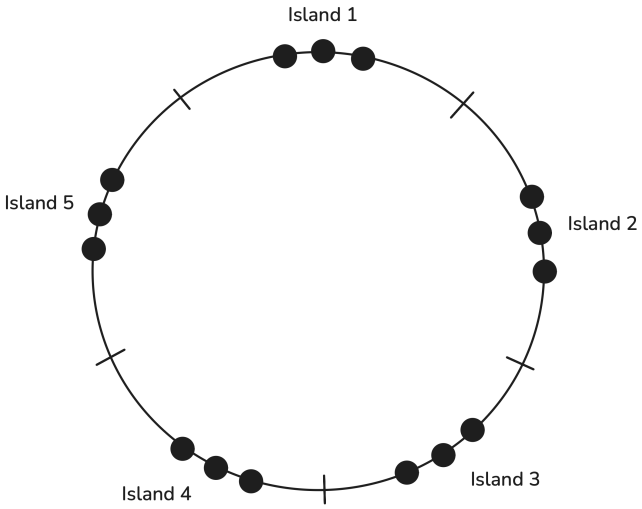

Figure 11: Kernel Ring Blocks

Since each island has a specific size, each is responsible for a corresponding set of chromosomes. By treating the chromosome list as a circular structure, where the last element wraps around to the first using the `%` operator, migration between islands can be implemented [8]. The migration process is visualized in Figure 12.

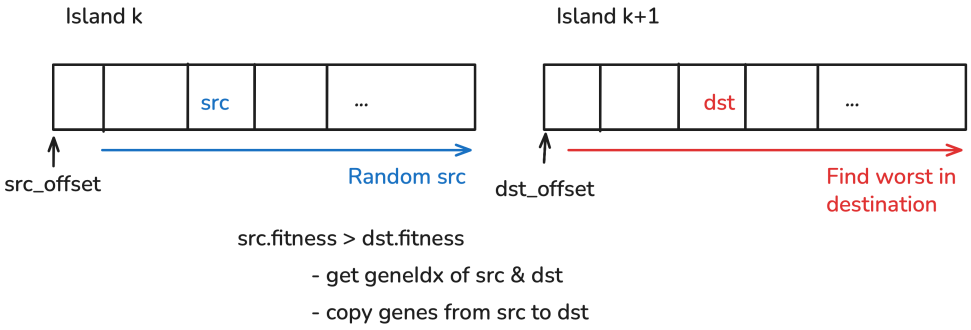

Figure 12: Migrate Kernel Workings

The migration step is performed from the point of view from island k and target island $k + 1$. Each island has its own offset, respectively identified by `src_offset` and `dst_offset`.

A random `src` index will be found using randomness. Accordingly, the chromosome with the worst fitness value in target island will be selected. If the fitness value of the `src` is greater compared to `dst`, the `geneIdx` are retrieved and the genes from `src` are copied to `dst`.

6. Analysis

This section will analyze the performance of the parallel GA implementation. Once this analysis is performed, an execution time comparison will be conducted against the sequential version.

6.1. Methodology

The timing data collected for the sequential version executed **5** runs per configuration. The accompanying charts for the Cov & Std dev can be found in Section 8.2.1.

For the parallel implementation the timing data collection was made using the `std::chrono` library inside of the program. For each program execution, **5** runs were collected. The accompanying charts for the Cov & Std dev can be found in Section 8.3.1.

Execution timing data for the sequential & parallel version are within guidelines.

Collection metrics for the parallel version regarding bandwidth, etc, was a bit more complicated, due to the number of kernels launched. For this reason the `ncu` [5] CLI was used in combination with `benchkit` [6]. This made it possible to collect targeted per-kernel metrics by applying predefined profiles to the selected metrics.

The profiled kernels are the following: `initBufferKernel`, `initIslandBufferKernel`, `gaIslandKernel` and `evaluateIsland`. These kernels are the most involved in the algorithm, based on previous Nvidia Nsight Compute analysis. The collected metrics for each kernel were based on the `detailed` set. Due to how much time each profiling run takes, the number of iterations was reduced compared to the timing dataset. For all parallel benchmarks, the number of threads per block was fixed to **512**.

[5] https://developer.nvidia.com/nsight-compute

[6] https://github.com/open-s4c/benchkit

6.2. Execution Time

Let's start by analyzing the execution time and the impact parameters have on the resulting fitness value. The execution time vs application parameters is shown in Figure 13.

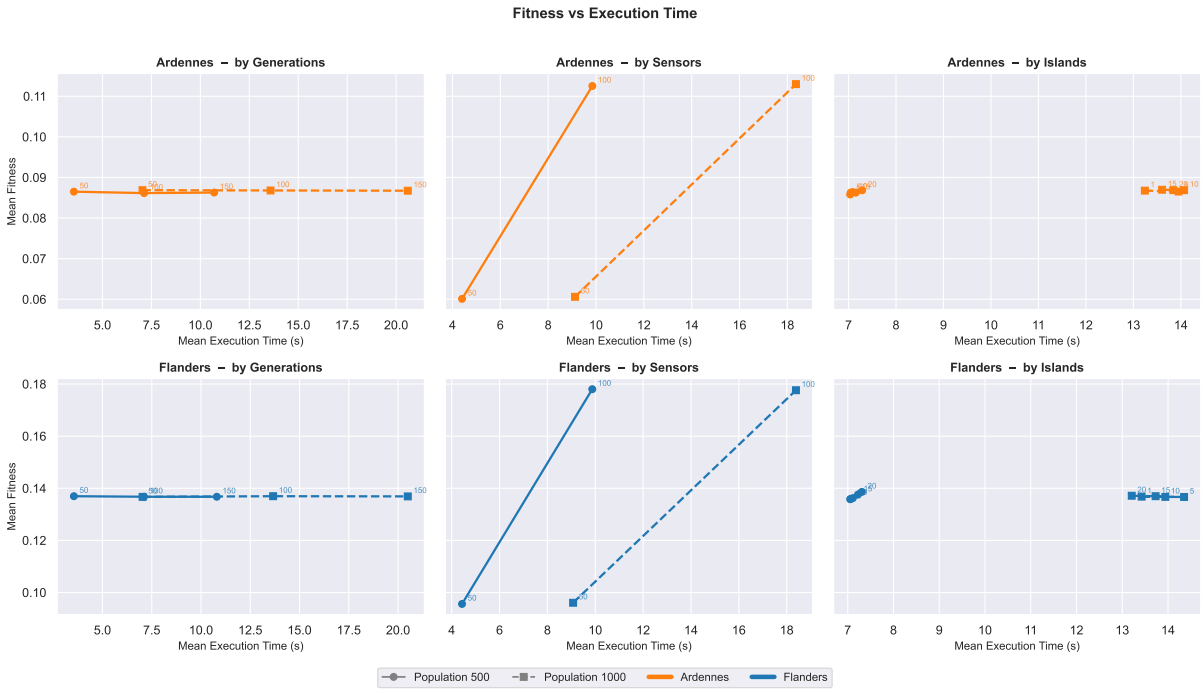

Figure 13: Timing Dataset - Fitness vs Time

The first distinction that becomes clear is that, between the terrain types, the Flanders area has a much higher fitness value. Changing the number of generations, regardless of population does not change the resulting fitness value while increasing the execution time substantially.

The number of islands has a negligible effect on the execution time for the smallest population of 500. There is a *very* small increase in fitness value & execution time when increasing the population size from 5 until 20. For the larger population, the situation is reversed; the highest number of islands (20), gives a slightly higher fitness value with slight reduction in execution time.

The parameter which has the highest impact on fitness & execution time is the number of sensors. This is to be expected, increasing the number of sensors, results in an increase of the coverage of the grid and a higher fitness value.

Plotting the execution time in function of algorithm parameters can be found in Figure 14.

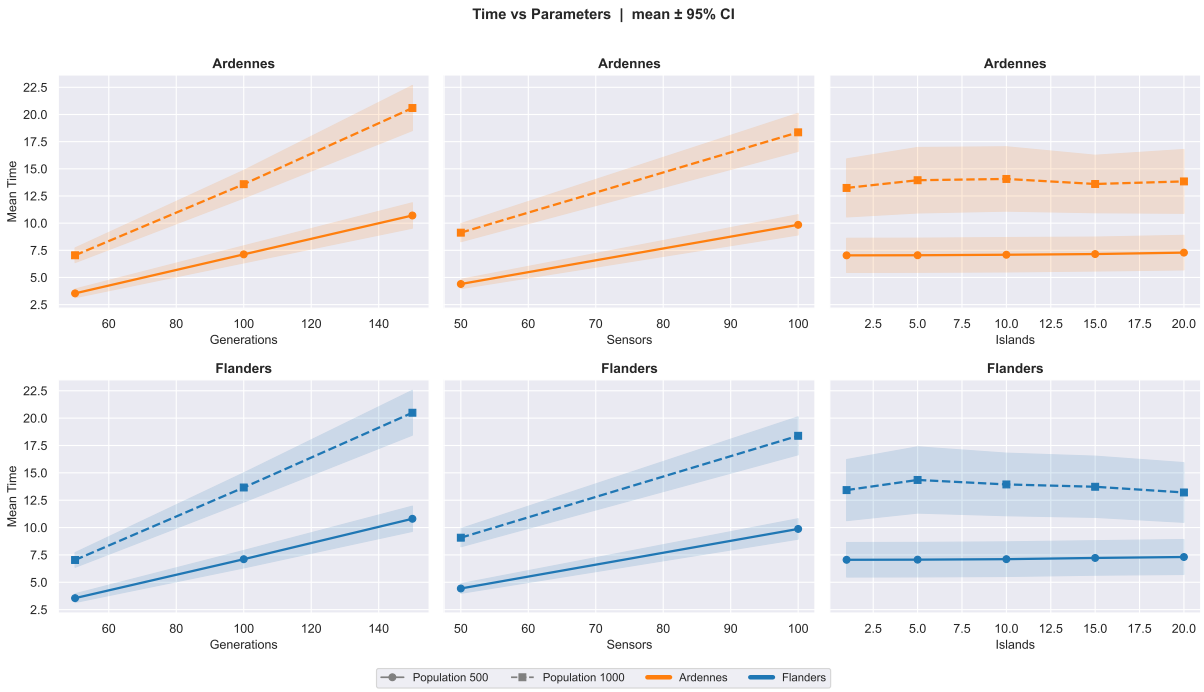

Figure 14: Timing Dataset - Time vs Parameters

The same behavior, as noted previously, is more defined here. Both the generations & sensors parameters have a negative impact on the execution time. The sensor parameter is the only one that results in an increase of the fitness value. For the island parameter, there is a slight increase and downward trend when increasing the number of islands.

In addition, a test was carried out, to check if increasing the population size to a much larger value, beyond 1000 would have any impact, regardless of previous behavior discussed. The analysis of this can be found in Figure 15. The population was varied with following fixed values: sensors: 100; generations: 50; islands: 10.

Figure 15: Large Population Dataset - Population Size vs Fitness

The charts make it conclusive; increasing the population size beyond 1000 does not have any effect on an increasing fitness value. The chart does again show the plain distinciton in fitness value between The Ardennes & Flanders region.

6.2.1. Conclusion

Definitive conclusions regrading this behavior require the more detailed analysis provided in Section 6.3. However, initial assessment suggests that the GA algorithm is already well optimized for the problem space.

6.3. Profiling

The analyzed kernels for this section are the following: `initBufferKernel`, `initIslandBufferKernel`, `galslandKernel` and `evaluateIsland`. This is based on previously made analysis using Nvidia Nsight Compute.

The following values are measured with the following parameter config:

- population: 1000
- sensors: 100
- generations: 50

To start, the execution time for each kernel vs #islands is shown in Figure 16. This timing is the accumulated duration of 50 generations!

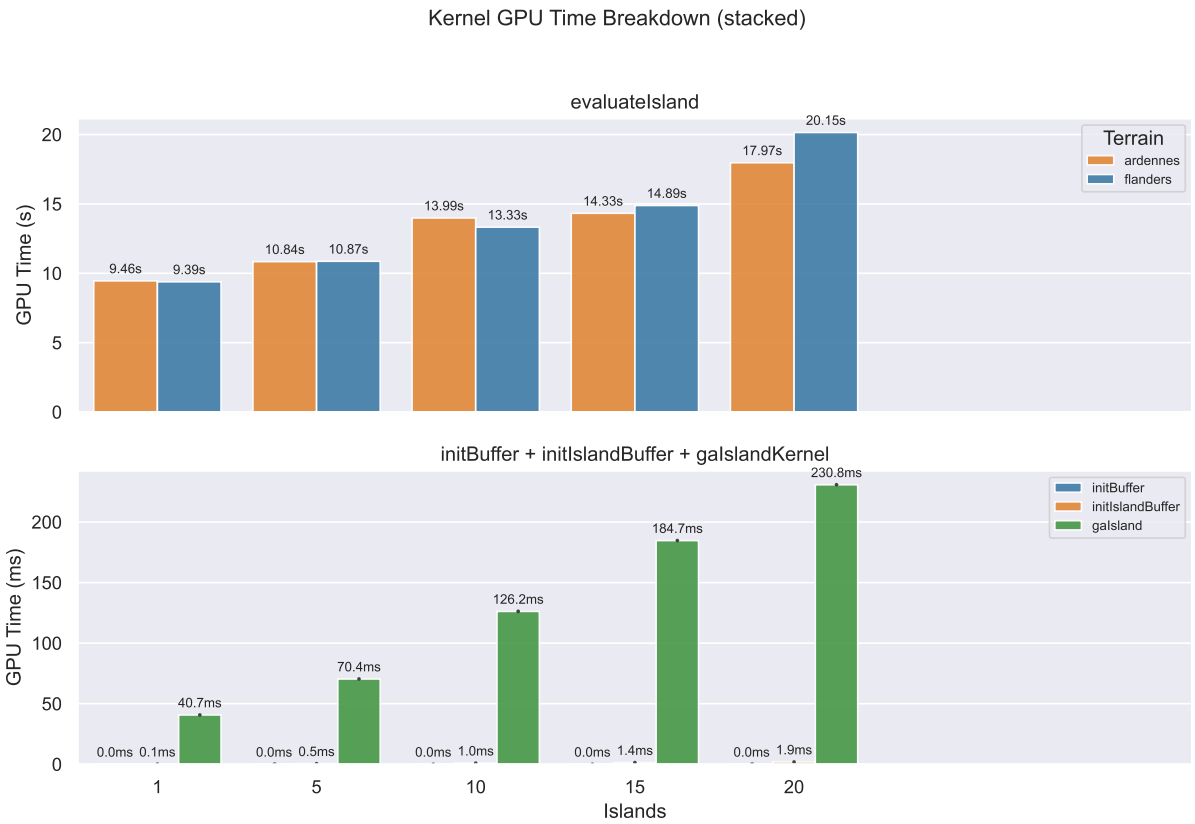

Figure 16: Kernel Execution Times vs Islands

From the outset, it is immediately clear that the major bottleneck is the `evaluateIsland` kernel. For this reason, the major focus will be on the kernels: `galsland` and `evaluateIsland`.

Each kernel's primary computation throughput is shown in Figure 17.

Figure 17: GFLOPs vs Islands

Both kernels are dominant in their respective operation type – the `galslandKernel` is dominantly single precision point heavy – while the `evaluateIsland` is dominated by double precision operations. In both kernels, it is clear that the throughput declines when increasing the number of islands. Before making any conclusion, the bandwidth per kernel must be analyzed, it could be that the kernels are memory bound.

To evaluate whether the kernels are memory- or compute-bound, the Roofline model was employed. The respective Roofline analyses for both FP32 and FP64 precision variants are presented in Figure 18 and Figure 19.

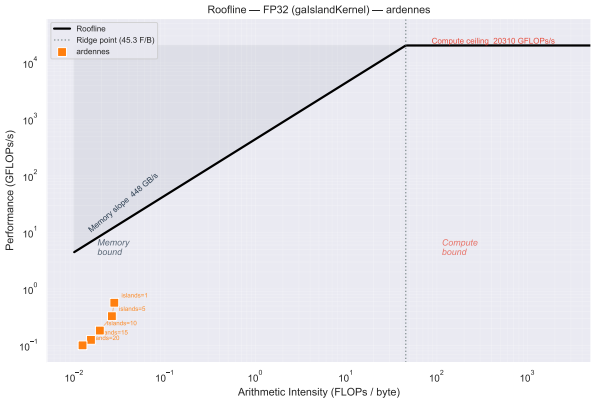

Figure 18: Roofline FP32 - `galslandKernel` Ardennes

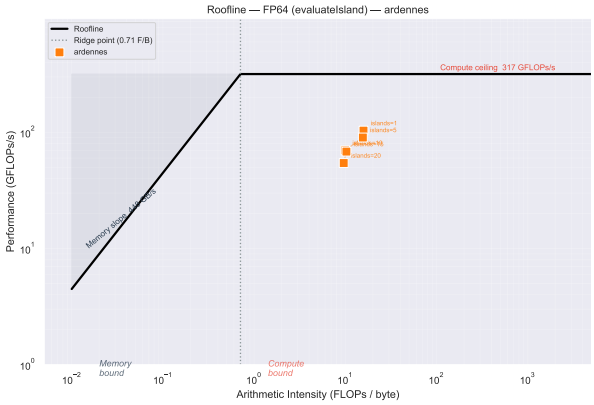

Figure 19: Roofline FP64 - `evaluateIsland` Ardennes

Based on both respective models, it can be concluded that neither kernel is either compute or memory bound. Before making a final determination, the warp occupancy percentage for both kernels is shown in Figure 20.

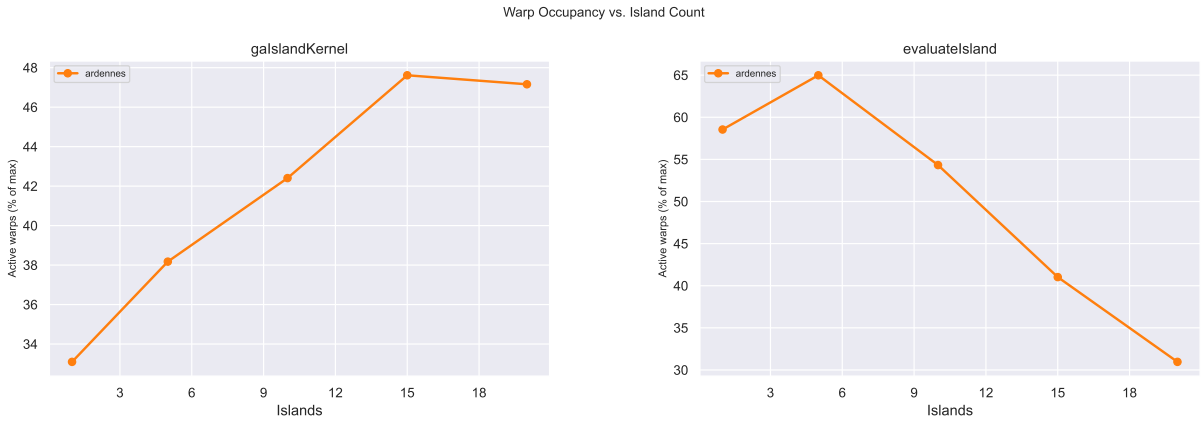

Figure 20: Warp Occupancy - Ardennes

The charts in Figure 20, showcase the warp occupancy of all the 46 SM on the RTX 3070 in percentage. For the `galslandKernel`, it shows that increasing the number of island increases the occupancy of warp's and leads to better utilization of available resources.

For the `evaluateIsland` it is a different story, increasing the number of islands shows a decrease in warp occupancy. The average occupancy level of the `evaluateIsland` is higher compared to the other kernel.

6.4. Vs Sequential

A comparison of the execution time & speedup between the sequential & parallel version is shown in Figure 21. Both comparisons are done with the same input configurations. For the GPU a mean is taken over all islands configurations.

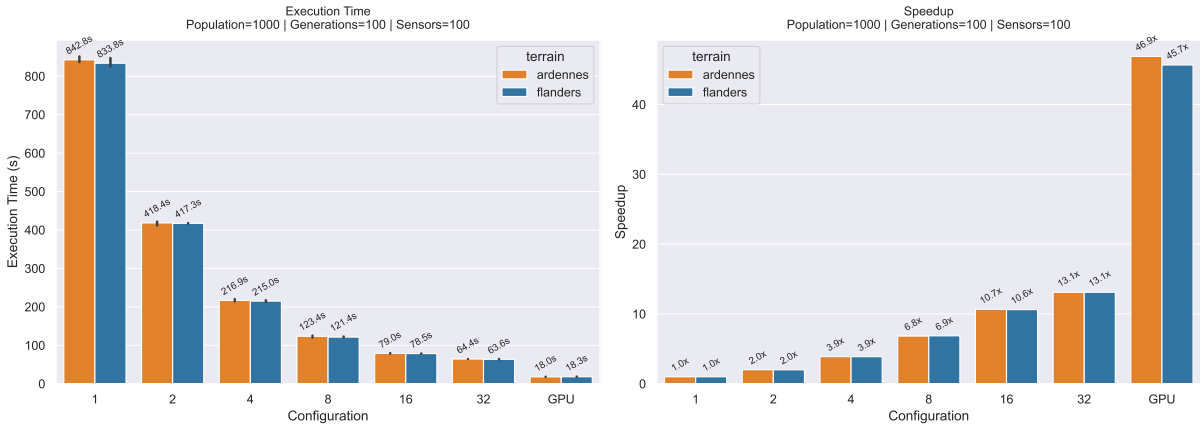

Figure 21: Sequential vs Parallel

The left chart in image Figure 21 displays the execution time of both versions, while the right chart shows the speedup. The speedup chart shows that, the naive application of OpenMP library on the sequential version delivers an impressive speedup between the 32 thread version and single thread version. The GPU based algorithm is substantially faster compared to the single threaded version. Proving the usage and implementation of GA algorithm on GPU based devices has clear benefit.

7. Conclusion

To summarize, both terrain exhibit distinctive fitness values which are expected based on the terrain features. No significant difference in execution time was observed between the terrains. The only factor impacting the fitness value is the #sensors which is reasonable. However, the lack of impact from the other parameters requires further investigation.

The throughput & bandwidth based kernels indicate that there is room for improvement on the GPU GA implementation. Nevertheless, as the analysis in Section 6.4 shows, even a relatively unoptimized GPU implementation achieves a significant speedup compared to both the single threaded and 32 threaded implementation.

The reduction in performance observed, when increasing the number of islands could possible be explained by the recreation of the `rawSharedData` value for each island in the `evaluateIsland` function. Future optimizations should look in to making this shared between all islands.

The `evaluateIsland` function remains the dominant kernel, even after transitioning to an island-based genetic algorithm (GA). Consequently, future optimization efforts should target this kernel. Transitioning from 64-bit floating-point (FP64) to 32-bit floating-point (FP32) operations represents a potential avenue for improvement, though it is worth noting that the current FP64 peak performance has not yet been reached.

Overall, the implementation was successful, and the parallel algorithm offers significant potential for further enhancements in efficiency.

8. Appendix

8.1. Platform

The benchmarks were executed on a KUbuntu 25.04 desktop, with the specifications listed in Table 1.

Part	Value
CPU	Ryzen 9 5950X
GPU	RTX 3070
Driver Version	595.97
RAM	64GB (3200 Mhz)
OS	Windows 11 - Version 10.0.22631 Build 22631
WSL Version	2
WSL Distro	24.04
Kernel Version	6.6.87.2-microsoft-standard-WSL2
NVCC	13.2, V13.2.51
NCU	Version 2026.1.0.0 (build 37166530)
GCC	gcc (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0
OpenMP	201511

Table 1: Desktop Specifications

8.2. Charts

8.2.1. Sequential

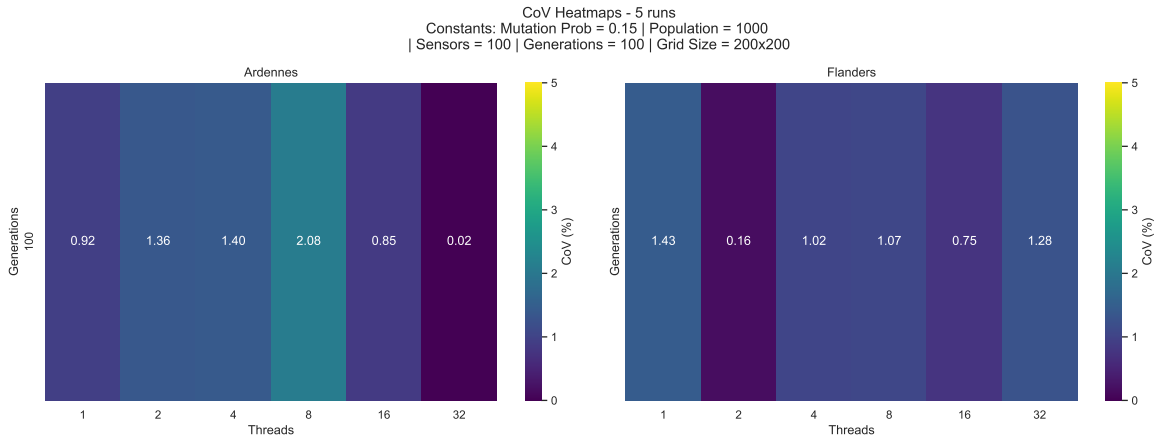

Figure 22: Sequential - Execution Time CoV

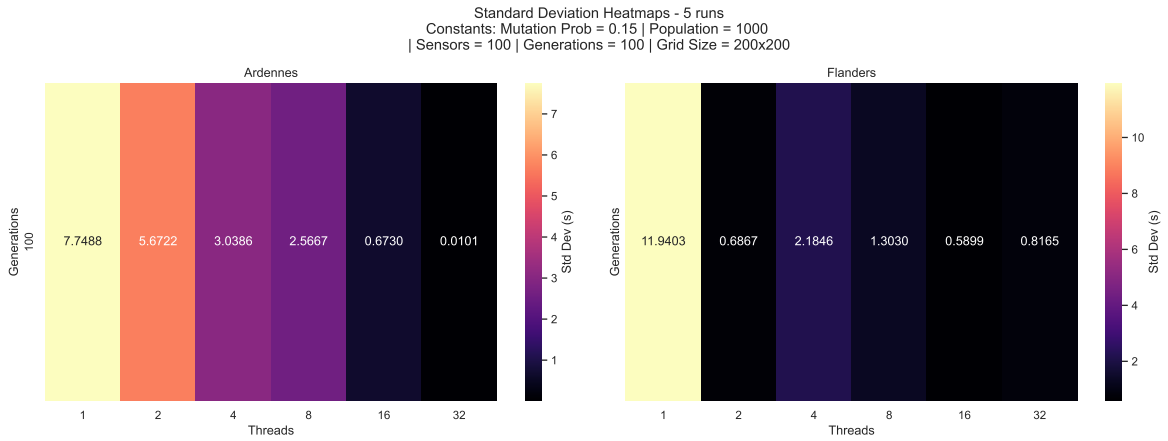

Figure 23: Sequential - Execution Time Std Dev

8.3. Parallel

8.3.1. Timing

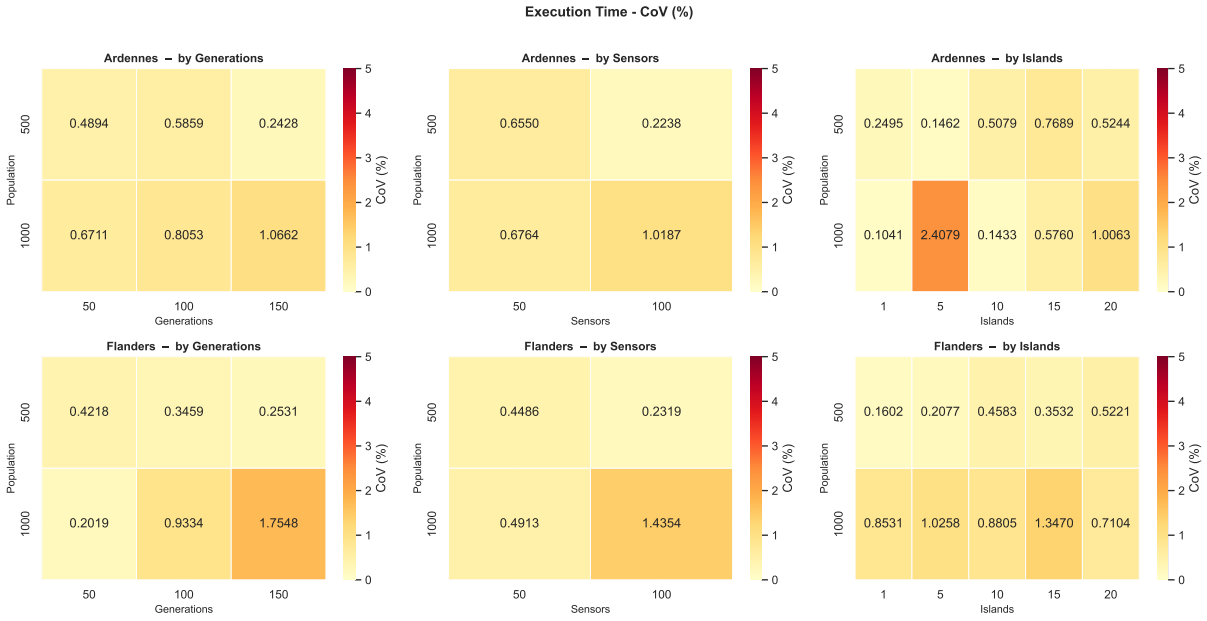

Figure 24: Timing Dataset - Execution Time CoV

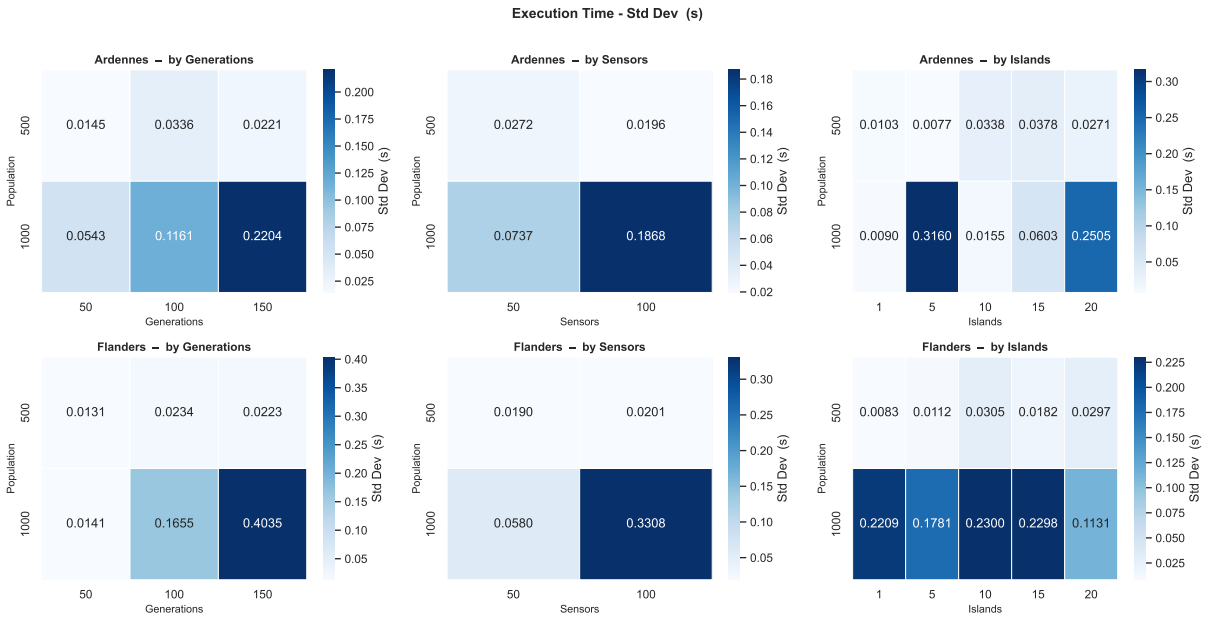

Figure 25: Timing Dataset - Execution Time Std Dev

8.3.2. Large Population

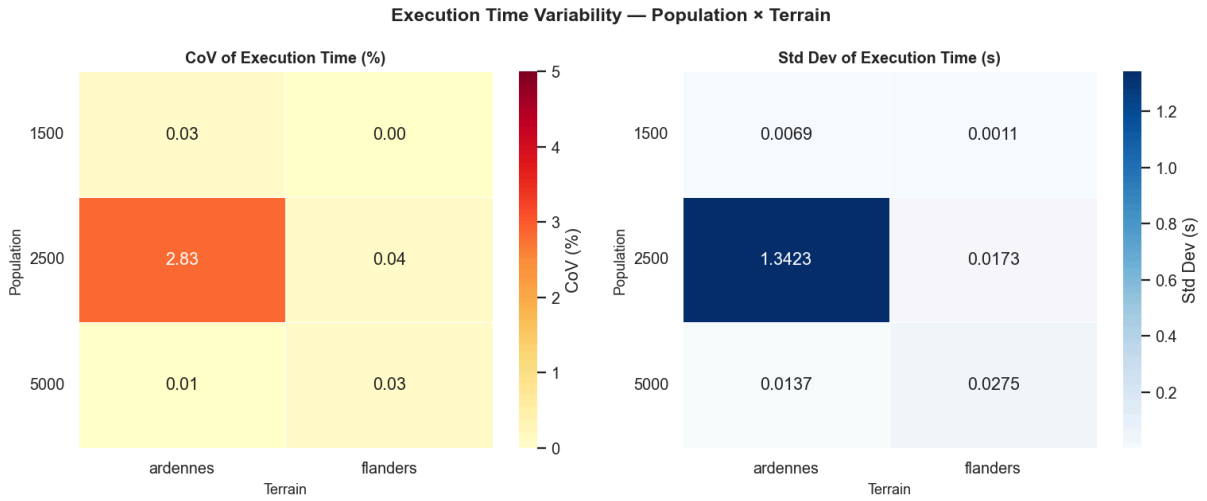

Figure 26: Large Population Dataset - Execution Time Cov & Std Dev

Bibliography

[1] M. Lagae, "DEM-Data-Preprocessing." [Online]. Available: https://github.com/gaMingLT/DEM-Data-Preprocessing

[2] B. Raymond Chee, "Final Project for 15-618 Parallel Computing ." [Online]. Available: https://github.com/rjchee/parallel_genetic_algorithms

[3] Y. Yoon and Y.-H. Kim, "An Efficient Genetic Algorithm for Maximum Coverage Deployment in Wireless Sensor Networks," *IEEE Transactions on Cybernetics*, vol. 43, no. 5, pp. 1473–1483, Oct. 2013, doi: 10.1109/TCYB.2013.2250955.

[4] J. P. Ridder and J. C. HandUber, "Mission Planning for Joint Suppression of Enemy Air Defenses Using a Genetic Algorithm," in *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation*, Washington DC USA: ACM, June 2005, pp. 1929–1936. doi: 10.1145/1068009.1068334.

[5] S. S. Dhillon and K. Chakrabarty, "Sensor Placement for Effective Coverage and Surveillance in Distributed Sensor Networks."

[6] J.-H. Seo, Y. Yoon, and Y.-H. Kim, "An Efficient Large-Scale Sensor Deployment Using a Parallel Genetic Algorithm Based on CUDA," *International Journal of Distributed Sensor Networks*, vol. 12, no. 3, p. 8612128, Mar. 2016, doi: 10.1155/2016/8612128.

[7] Frank Warmerdam Even Rouault and others, "GDAL." [Online]. Available: https://gdal.org/en/stable/

[8] J. R. Cheng and M. Gen, "Accelerating Genetic Algorithms with GPU Computing: A Selective Overview," *Computers & Industrial Engineering*, vol. 128, pp. 514–525, Feb. 2019, doi: 10.1016/j.cie.2018.12.067.